Part 3 — Standard Nim: Clear Explanation and Python Implementation

Let's understand the rules first, work through the example `(3, 2)`, determine the winning strategy, and then write the code for all three required functions.

1. What is Standard Nim?

Nim is a two-player game in which players take turns removing objects from piles. The player who removes the last object wins.

Suppose we have two piles:

- Pile 1 contains 3 objects.
- Pile 2 contains 2 objects.

We represent the game state as:

```
(3, 2)
```

The first number represents the size of the first pile, and the second number represents the size of the second pile.

For this assignment, you can remove **any positive number of objects from exactly one non-empty pile per turn**. You cannot remove objects from both piles in the same move.

For example, from `(3, 2)`:

Move	Result	Legal?
Remove 1 object from the first pile	`(2, 2)`	Yes
Remove 2 objects from the first pile	`(1, 2)`	Yes
Remove all 3 objects from the first pile	`(2,)`	Yes
Remove 1 object from the second pile	`(3, 1)`	Yes

Move	Result	Legal?
Remove both objects from the second pile	`(3,)`	Yes
Remove objects from both piles	`(2, 1)`	No

Why is `(3,)` used instead of `(3, 0)` ?

Because the assignment requires us to **omit zero-sized piles**. Therefore, once a pile becomes empty, we remove it from the tuple.

The states should also be sorted in non-increasing order. For example:

```
(1, 3)  →  (3, 1)
```

This makes the representation consistent. We do not need to treat `(3, 1)` and `(1, 3)` as different states because they describe the same collection of piles.

---

2. What Does Winning or Losing Mean?

Your assignment defines the function `nim_is_winning(state)` to return a Boolean value.

- `True` : The player whose turn it is has a strategy that guarantees a win.
- `False` : The player whose turn it is will lose if the opponent also plays optimally.

The important part is **optimal play**. We assume that neither player makes a mistake.

The base case: An empty state

Consider:

```
()
```

There are no objects left, so the game has ended.

The previous player removed the last object and won. Therefore, the player whose turn it would be next has no legal move and loses.

Thus:

```
nim_is_winning(()) == False
```

This is the base case for our recursive algorithm.

How does the recursive strategy work?

Suppose the current state is `(1,)` .

There is only one legal move:

```
(1,) → ()
```

The next player receives an empty state, which is losing.

Therefore, the current player can make a move that leaves the opponent in a losing state.

Consequently:

```
nim_is_winning((1,)) == True
```

This gives us the central rule:

> A state is winning if at least one legal move leads to a losing state for the opponent. Otherwise, it is losing.

In Python, this rule becomes:

```
for next_state in nim_successors(state):
    if not nim_is_winning(next_state):
        return True

return False
```

We will use this rule to solve the game recursively.

---

3. Understand the Three Required Functions

Your starter code contains three functions to implement.

Function 1: `nim_successors(state)`

Purpose: Generate every distinct legal next state.

For example, the input:

```
nim_successors((3, 2))
```

should produce:

```
(3, 2) → (2, 2)
(3, 2) → (1, 2)
(3, 2) → (2,)
(3, 2) → (3, 1)
(3, 2) → (3,)
```

The order of these states in the final output will follow the sorting rule we implement.

The function works by:

1. Looking at each pile.
2. Trying every possible positive number of objects to remove from that pile.
3. Leaving all other piles unchanged.
4. Removing any pile that becomes zero.
5. Sorting the remaining pile sizes in non-increasing order.
6. Removing duplicate states, if any.

Function 2: `nim_is_winning(state)`

Purpose: Determine whether the current state is winning or losing.

It uses recursion:

1. If there are no objects, return `False`.
2. Generate all legal next states.
3. Check whether any next state is losing for the opponent.
4. If one exists, return `True`.
5. If every next state is winning for the opponent, return `False`.

We also use `@lru_cache(maxsize=None)`.

This stores the answers to previously evaluated states, so the program does not need to solve the same state repeatedly.

Function 3: `nim_best_moves(state)`

Purpose: Return all legal moves that leave the opponent in a losing state.

For example, if a state has three possible moves but only one of them leaves the opponent with a losing position, this function returns that one move.

The result is a list of states, not a list of descriptions such as "remove two objects from pile 1."

---

4. Predict the Result Before Coding

Let's predict the answer for your starting state:

```
initial_nim_state = (3, 2)
```

The legal moves are:

- `(2, 2)`
- `(1, 2)`
- `(2,)`
- `(3, 1)`
- `(3,)`

We need to determine which of these leave the opponent in a losing state.

A useful mathematical property of standard Nim is the **nim-sum**, calculated by taking the bitwise XOR of the pile sizes.

For `(3, 2)` :

[
3 \oplus 2 = 1
]

Because the nim-sum is nonzero, the position is winning for the player whose turn it is.

The optimal strategy is to make the nim-sum zero.

For example, removing one object from the pile of size 3 produces:

[
(3,2)\rightarrow(2,2)
]

Now:

[
2\oplus2=0
]

The opponent receives a losing position under optimal play.

Therefore, removing one object from the pile of size 3 **is an optimal move**.

There is another winning move: removing one object from the pile of size 2.

[
(3,2)\rightarrow(3,1)
]

Since:

[
3\oplus1=2
]

this particular calculation is nonzero, so it does not produce a losing position. Thus, this move is not winning for the player who moves first.

The code will verify all legal moves rather than relying on intuition alone.

---

5. Complete Python Code

The code below implements all three functions from your starter code. It follows the assignment's rules: remove any positive number of objects from exactly one pile, omit empty piles, and sort states in non-increasing order.

from functools import lru_cache

def nim_successors(state):
moves = set()

```
for i in range(len(state)):
    for remaining in range(state[i]):
        next_state = list(state)
```

```
        next_state[i] = remaining

        next_state = tuple(
            sorted(
                (pile for pile in next_state if pile > 0),
                reverse=True
            )
        )

        moves.add(next_state)

return sorted(moves, reverse=True)
```

```
@lru_cache(maxsize=None)
def nim_is_winning(state):
if len(state) == 0:
return False

for next_state in nim_successors(state):
    if not nim_is_winning(next_state):
        return True

return False

def nim_best_moves(state):
winning_moves = [ ]

for next_state in nim_successors(state):
    if not nim_is_winning(next_state):
        winning_moves.append(next_state)

return winning_moves

initial_nim_state = (3, 2)

print('Initial state:', initial_nim_state)
print('Legal moves:', nim_successors(initial_nim_state))
print('Winning for player to move:', nim_is_winning(initial_nim_state))
print('Winning moves:', nim_best_moves(initial_nim_state))

test_states = [(1,), (1, 1), (2, 2), (1, 2)]

print('\nTest results:')
```

```
for state in test_states:
print(
'State:', state,
'| Winning:', nim_is_winning(state),
'| Winning moves:', nim_best_moves(state)
)
```

Expected output

```
Initial state: (3, 2)
Legal moves: [(3,), (3, 1), (2, 2), (2, 1), (2,)]
Winning for player to move: True
Winning moves: [(2, 2)]

Test results:
State: (1,) | Winning: True | Winning moves: [()]
State: (1, 1) | Winning: False | Winning moves: []
State: (2, 2) | Winning: False | Winning moves: []
State: (1, 2) | Winning: True | Winning moves: [(1, 1)]
```

Why does the output contain `(2, 1)` instead of `(1, 2)` ?

The code sorts every state in non-increasing order, so `(1, 2)` is represented as `(2, 1)` . Both describe the same pile sizes.

Also, the empty state is represented by `()` , which is a valid Python tuple.

---

6. Analyse the results

Let's understand why each test state is winning or losing.

State 1: `(3, 2)`

Result: Winning (`True`)

The player can remove one object from the pile of size 3:

[
(3,2)\rightarrow(2,2)
]

The opponent receives `(2, 2)`, which is a losing state.

Therefore, `(2, 2)` is a winning move for the current player.

State 2: `(1,)`

Result: Winning (`True`)

There is only one possible move:

[
(1,)\rightarrow()
]

The current player removes the last object and wins. The opponent receives an empty state and loses.

Therefore, `(1,)` is winning.

State 3: `(1, 1)`

Result: Losing (`False`)

The current player has two possible moves, but both are equivalent:

[
(1,1)\rightarrow(1,)
]

The opponent receives one object and can remove it to win.

No matter which pile the current player chooses, the opponent can win.

Therefore, `(1, 1)` is losing.

State 4: `(2, 2)`

Result: Losing (`False`)

Every legal move leaves the opponent with an unequal pair of piles or a single non-empty pile.

For example:

[
(2,2)\rightarrow(2,1)
]

The opponent can then move to `(1, 1)`, leaving the original player in a losing position.

Similarly, removing both objects from one pile produces `(2,)`, which is winning for the opponent.

Therefore, `(2, 2)` is losing under optimal play.

State 5: `(1, 2)`

Result: Winning (`True`)

The player can remove one object from the pile of size 2:

[
(2,1)\rightarrow(1,1)
]

The opponent receives `(1, 1)`, which is losing.

Therefore, `(1, 1)` is the winning move.

---

7. Final assignment table

You can fill in your assignment table as follows:

Starting state	Winning or losing for player to move?	Winning move(s)
`(3, 2)`	Winning (`True`)	`(2, 2)`
`(1,)`	Winning (`True`)	`()`
`(1, 1)`	Losing (`False`)	None
`(2, 2)`	Losing (`False`)	None
`(1, 2)`	Winning (`True`)	`(1, 1)`

The winning moves shown are the resulting states, not descriptions of which pile to choose.

For example, `(2, 2)` means removing one object from the pile of size 3 in the initial state `(3, 2)`.

---

8. Explanation to write in your assignment

Standard Nim is a two-player game in which each player removes a positive number of objects from exactly one non-empty pile per turn. The player who removes the last object wins. Each game state is represented as a tuple of pile sizes, sorted in non-increasing order, with empty piles omitted.

The nim_successors() function generates all distinct legal next states. The nim_is_winning() function recursively determines whether the current player has a winning strategy. An empty state is losing because no legal moves remain. A state is winning if at least one legal move leads to a losing state for the opponent. The nim_best_moves() function returns all moves that leave the opponent in a losing state.

For the initial state (3, 2), the player can remove one object from the pile of size 3, resulting in (2, 2). This is a losing state for the opponent. Therefore, the initial state is winning, and (2, 2) is the winning move.

The tests show that (1,) and (1, 2) are winning states, while (1, 1) and (2, 2) are losing states. The recursive solver uses caching to avoid repeatedly evaluating the same states.

9. Important concepts to remember

Concept	Meaning
`nim_successors()`	Generates all distinct legal moves.
`nim_is_winning()`	Determines whether the current player can guarantee a win.
`nim_best_moves()`	Returns moves that leave the opponent in a losing state.
`()`	Empty state; the player whose turn it is loses.
`True`	The current player has a winning strategy.
`False`	The current player loses under optimal play.
`@lru_cache`	Stores previously calculated results to avoid repeated work.
`sorted(..., reverse=True)`	Sorts pile sizes in non-increasing order.

One clarification about your assignment: The introductory Standard Nim rules allow removing *any positive number* of objects from a pile. A later illustrative section mentions a maximum of two objects per move, but that describes a different variant. The implementation above follows the explicit Standard Nim requirements for Task 3.

Final answer

For the initial state `(3, 2)`:

- **Is it winning?** Yes (`True`).
- **Winning move:** `(3, 2)` → `(2, 2)`.
- **Why?** The move leaves the opponent with a losing position, so MAX can force a win with optimal play.